



Lecture 5 — REST API Design & FastAPI Fundamentals

AI-Assisted Software Development and Intelligent Applications

Today's Agenda

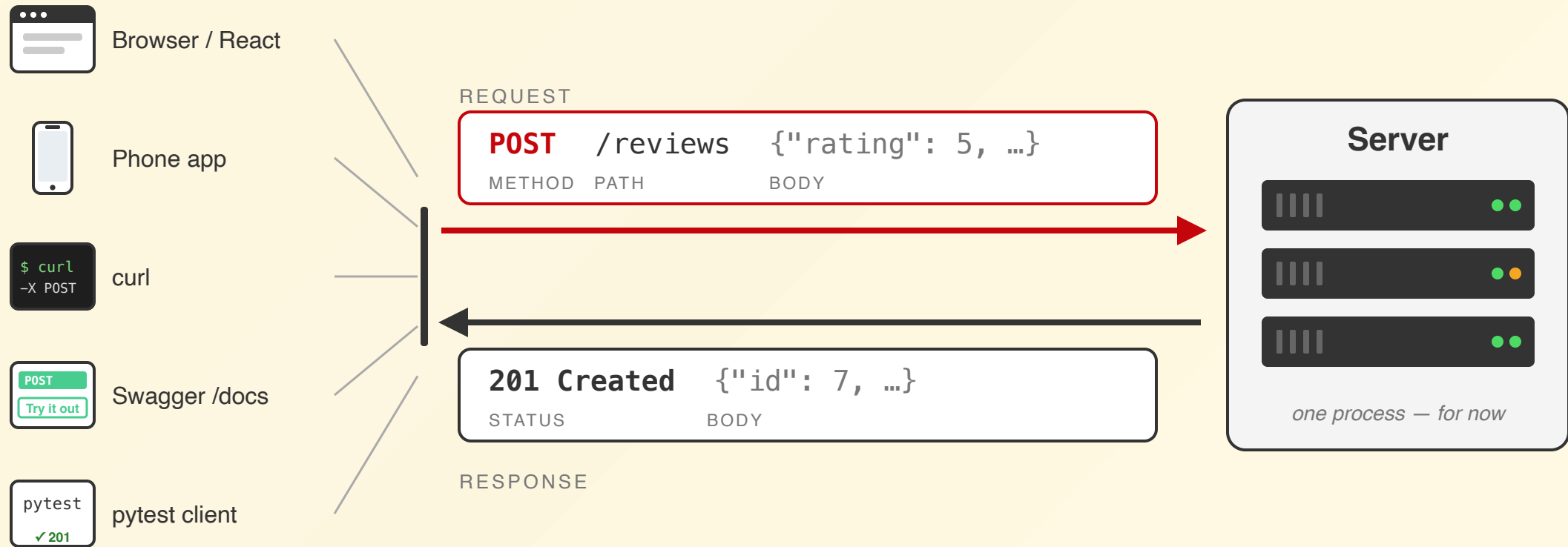
1. Client–server basics: HTTP methods, CRUD, status codes
2. Designing an API contract before writing code
3. FastAPI fundamentals
4. **Activity** — design `POST /reviews`
5. Many servers — why REST is stateless
6. WebSockets — when the connection must stay open
7. Thin vs. full client — what offline-first changes

Pre-Class Checklist — Start the Quiz

Did you complete the reading?

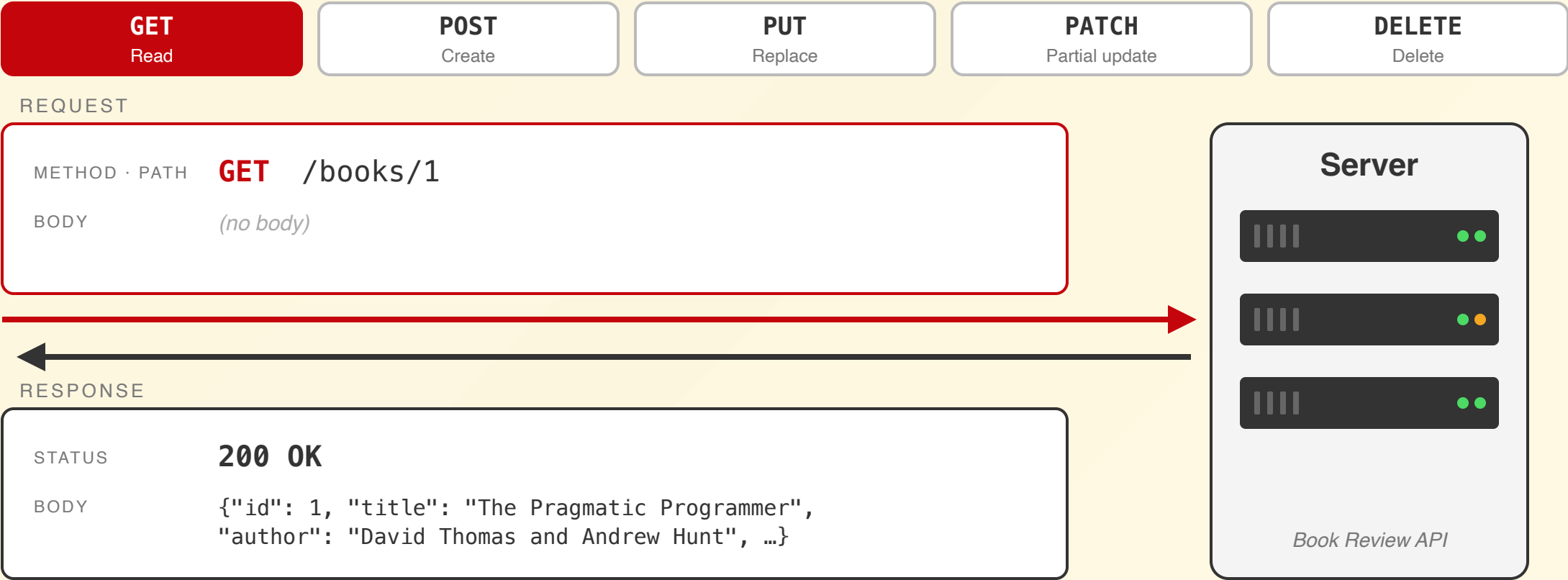
- HTTP methods (GET/POST/PUT/PATCH/DELETE) and CRUD mapping
- FastAPI tutorial — routes, Pydantic models, type hints, Swagger docs

Client and Server



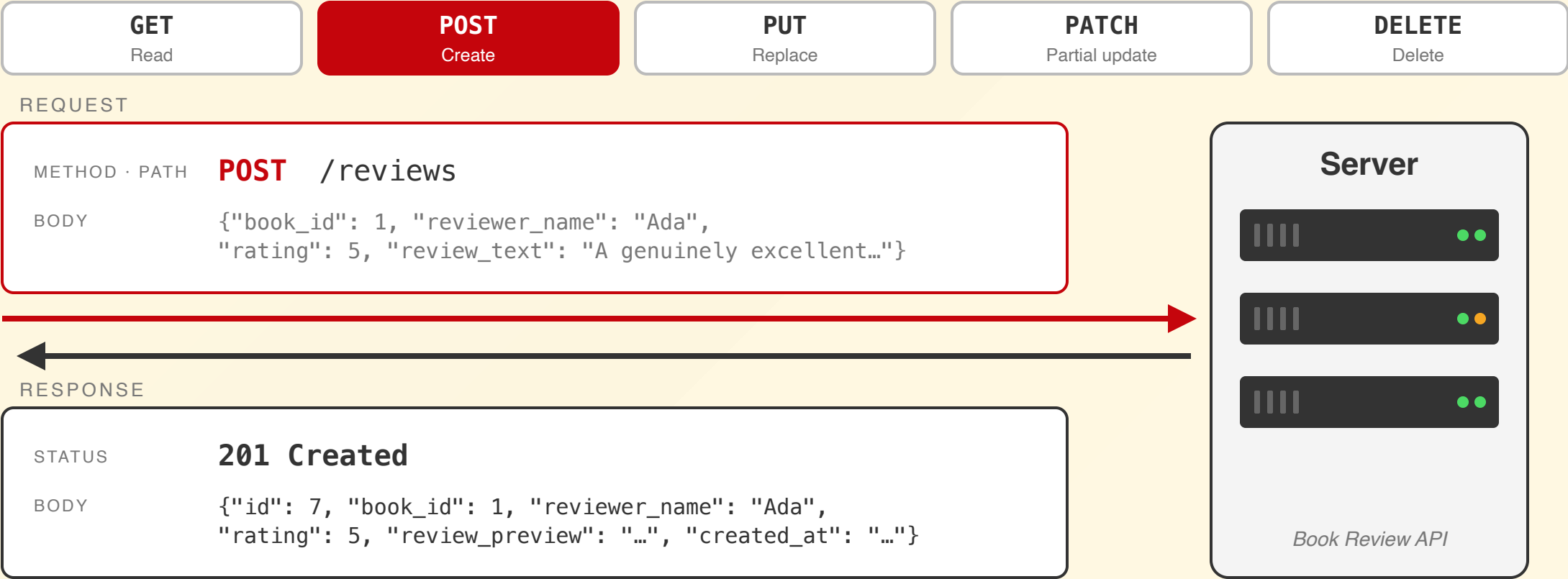
Then the exchange is over — the server remembers **nothing** about the client.

HTTP Methods → CRUD



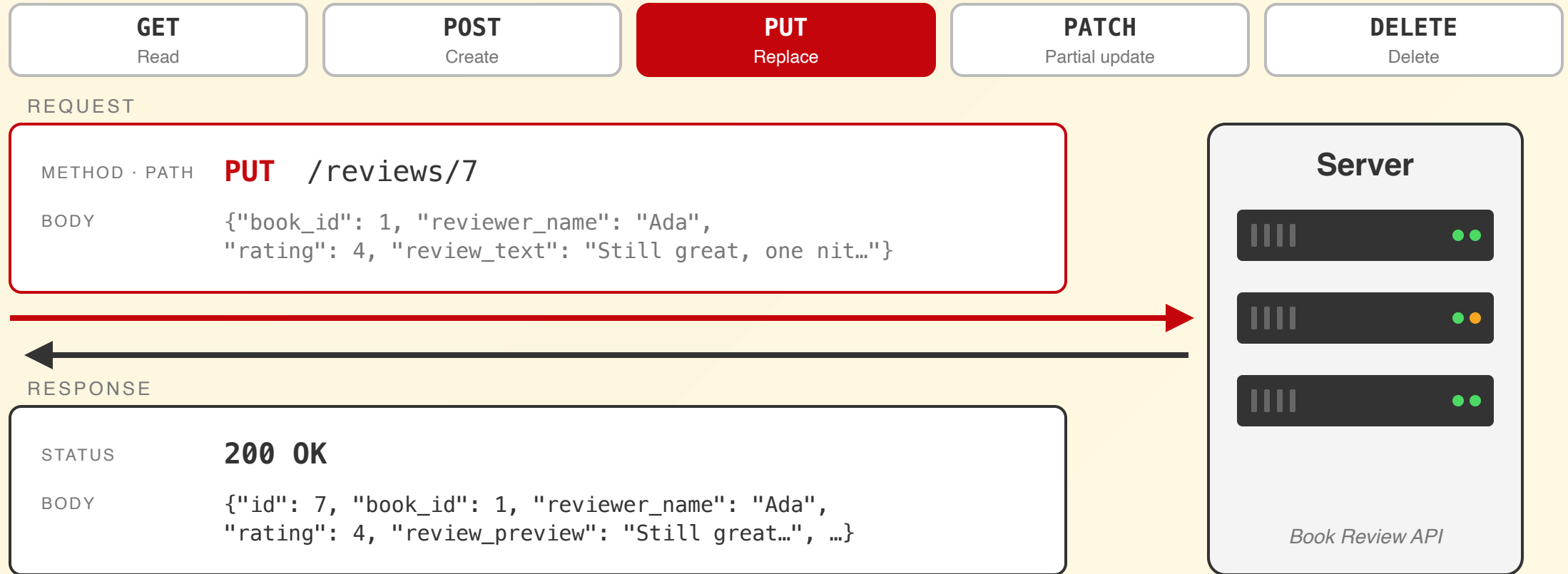
Safe and idempotent — ask 100 times, same answer, nothing changes.

HTTP Methods → CRUD



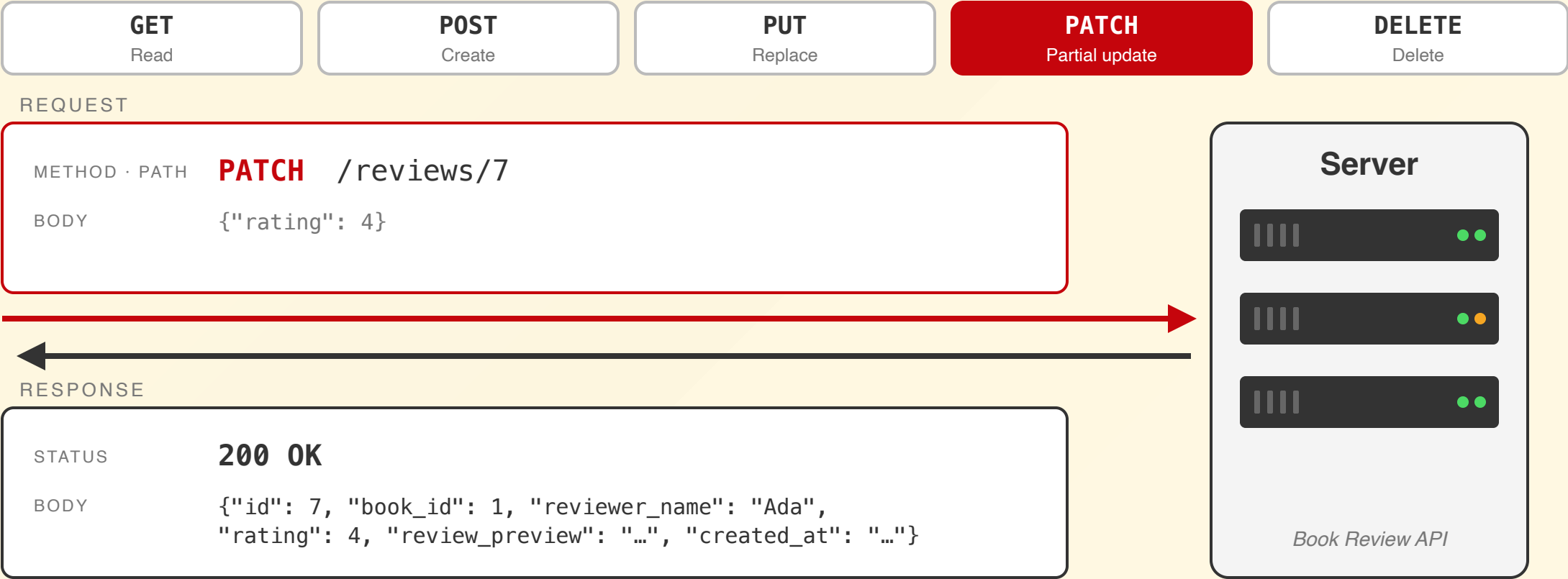
Not idempotent — send it twice and you have two reviews.

HTTP Methods → CRUD



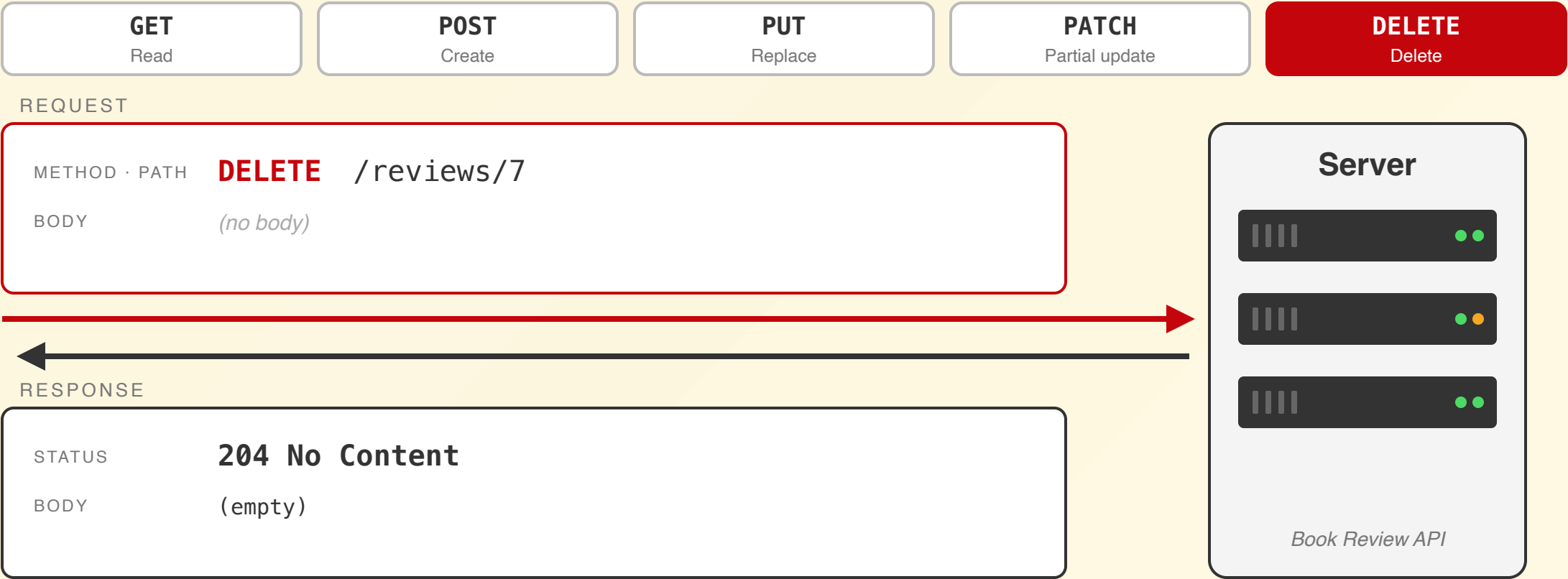
Replaces the **whole** resource — leave a field out and it's gone.

HTTP Methods → CRUD



Changes **only** the fields you send — everything else stays.

HTTP Methods → CRUD



Idempotent — deleting it again changes nothing; it's already gone.

Status Codes That Matter

2xx — it worked • 5xx — the *server* failed

Code	Meaning
200 OK	Success (GET, PUT, PATCH)
201 Created	A new resource now exists (POST)
204 No Content	Success, nothing to send back (DELETE)
500	Server bug — not the client's fault
503	Server down or overloaded — retry

4xx — the *client* got something wrong

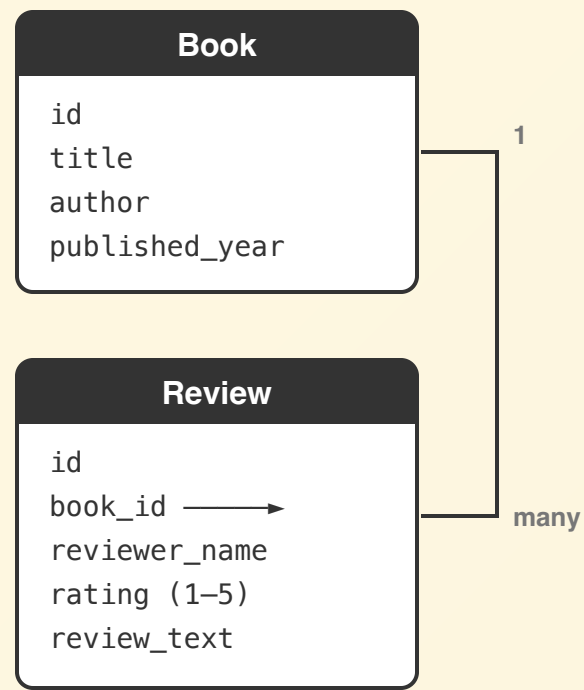
Code	Meaning
400	Malformed request
401	Not authenticated
403	Authenticated, but not allowed
404	Resource doesn't exist
422	Well-formed, but invalid data
429	Rate-limited — back off, retry later

Buzzer Checkpoint

**Launch Question 5 – discuss answers, then
continue**

Think Like an Architect

WHAT ARE THE OBJECTS?



a review belongs to exactly one book

Nouns become resources.

HOW DO THEY CHANGE?

Review

- created by a reader POST /reviews
- read, all or by book GET /reviews
- edited by author PATCH /reviews/{id}
- removed by author DELETE /reviews/{id}

Book

- read, all or one GET /books/{id}

no POST /books — the catalog is seeded, not posted

Book barely changes. Review has a whole life.
That asymmetry is a design fact, not an accident.

Changes become methods.

WHAT CAN GO WRONG?

- the book doesn't exist 404
- rating isn't 1–5 422
- a required field is missing 422
- it isn't your review 403
- our own code blew up 500

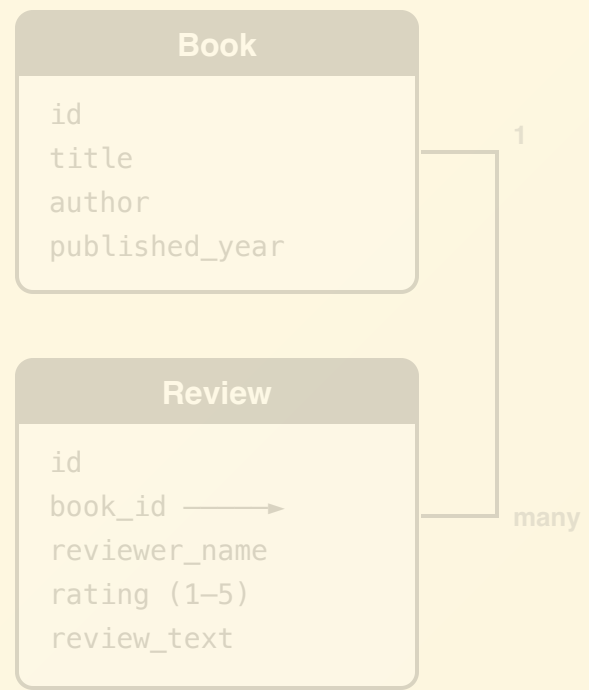
403 needs auth — that's Lecture 8.

This column is the one a first draft
always skips. Ask for it — sycophant challenge.

What goes wrong becomes status codes.

Think Like an Architect

WHAT ARE THE OBJECTS?



a review belongs to exactly one book

Nouns become resources.

HOW DO THEY CHANGE?

Review

- created by a reader POST /reviews
- read, all or by book GET /reviews
- edited by author PATCH /reviews/{id}
- removed by author DELETE /reviews/{id}

Book

- read, all or one GET /books/{id}

no POST /books — the catalog is seeded, not posted

Book barely changes. Review has a whole life.
That asymmetry is a design fact, not an accident.

Changes become methods.

WHAT CAN GO WRONG?

- the book doesn't exist 404
- rating isn't 1–5 422
- a required field is missing 422
- it isn't your review 403
- our own code blew up 500

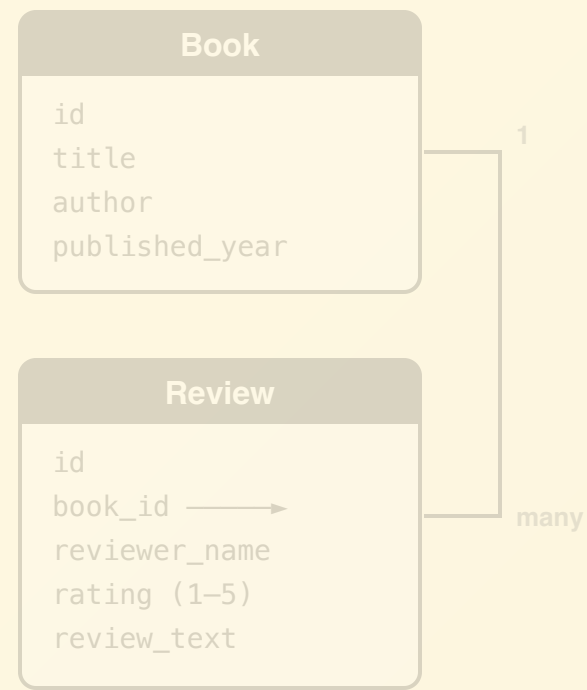
403 needs auth — that's Lecture 8.

This column is the one a first draft
always skips. Ask for it — sycophant challenge.

What goes wrong becomes status codes.

Think Like an Architect

WHAT ARE THE OBJECTS?



a review belongs to exactly one book

Nouns become resources.

HOW DO THEY CHANGE?

Review

- created by a reader POST /reviews
- read, all or by book GET /reviews
- edited by author PATCH /reviews/{id}
- removed by author DELETE /reviews/{id}

Book

- read, all or one GET /books/{id}

no POST /books — the catalog is seeded, not posted

Book barely changes. Review has a whole life.

That asymmetry is a design fact, not an accident.

Changes become methods.

WHAT CAN GO WRONG?

the book doesn't exist	404
rating isn't 1–5	422
a required field is missing	422
it isn't your review	403
our own code blew up	500

403 needs auth — that's Lecture 8.

This column is the one a first draft always skips. Ask for it — sycophant challenge.

What goes wrong becomes status codes.

FastAPI: Routes

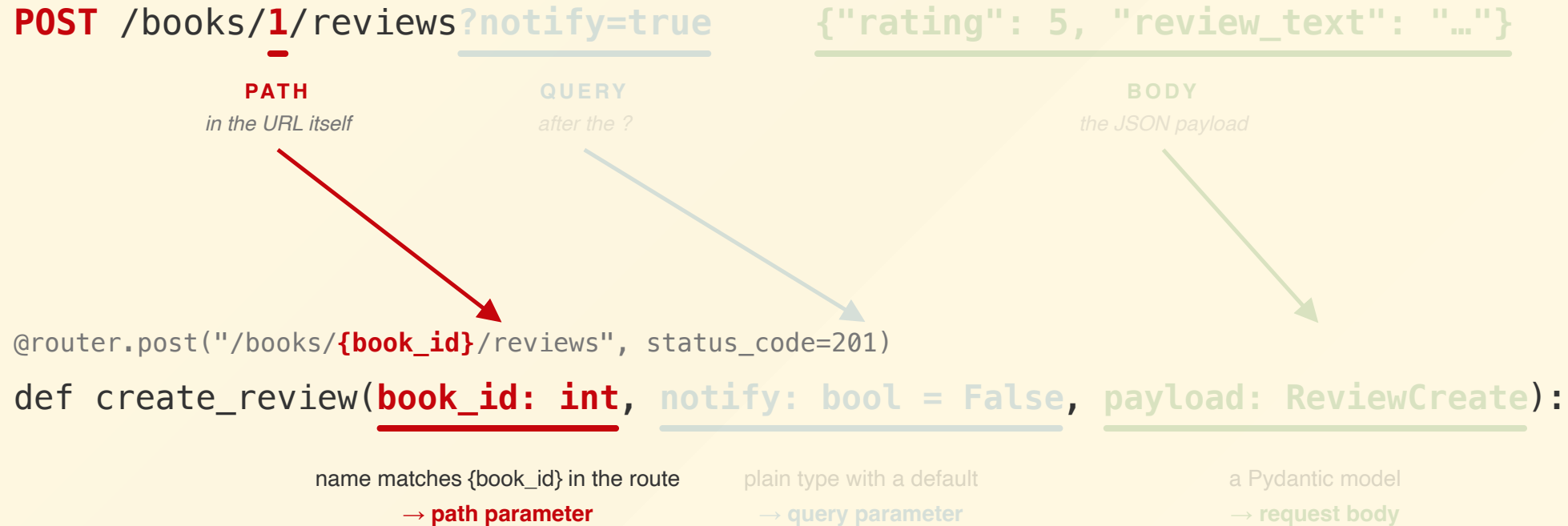
```
router = APIRouter(prefix="/books", tags=["books"])

@router.get("")
def get_books():
    ...

@router.get("/{book_id}")
def get_book(book_id: int):
    ...
```

Path parameters are type-annotated — FastAPI validates and converts them automatically.

Path, Query, Body



FastAPI decides where each argument comes from by its **name** and **type** — never by its position.

Path, Query, Body

POST /books/1/reviews?notify=true

PATH
in the URL itself

QUERY
after the ?

{"rating": 5, "review_text": "..."}
BODY

BODY
the JSON payload

@router.post("/books/{book_id}/reviews", status_code=201)

def create_review(book_id: int, notify: bool = False, payload: ReviewCreate):

name matches {book_id} in the route
→ **path parameter**

plain type with a default
→ **query parameter**

a Pydantic model
→ **request body**

FastAPI decides where each argument comes from by its **name** and **type** — never by its position.

Path, Query, Body

POST /books/1/reviews?notify=true

PATH
in the URL itself

QUERY
after the ?

{"rating": 5, "review_text": "..."}
BODY
the JSON payload

@router.post("/books/{book_id}/reviews", status_code=201)

def create_review(book_id: int, notify: bool = False, payload: ReviewCreate):

name matches {book_id} in the route
→ **path parameter**

plain type with a default
→ **query parameter**

a Pydantic model
→ **request body**

FastAPI decides where each argument comes from by its **name** and **type** — never by its position.

FastAPI: Pydantic Models

```
from pydantic import BaseModel, Field

class ReviewCreate(BaseModel):
    book_id: int
    reviewer_name: str
    rating: int = Field(ge=1, le=5)
    review_text: str
```

Request bodies are validated **before** your function runs — invalid data never reaches your code; FastAPI returns **422** automatically.

FastAPI: Response Models

```
@router.post("", response_model=ReviewResponse, status_code=201)
def create_review(payload: ReviewCreate):
    ...
```

`response_model` shapes and validates what goes *out*, independent of your internal data — and documents the response shape in Swagger automatically.

Swagger, for Free

Every FastAPI app gets interactive API docs at `/docs`, generated from your routes and Pydantic models — no separate documentation step. Use it today to verify your endpoint matches your design.

Buzzer Checkpoint

**Launch Question 6 – discuss answers, then
continue**

Python Tooling: Ruff

- `ruff check` — lint (unused imports, style issues, common bugs)
- `ruff format` — auto-format, the Python equivalent of Prettier

Run both locally before pushing — same habit as `npm run lint / format` from Lecture 2.

Activity — Design `POST /reviews`

Continuing the Book Review arc. `truncate_review_text()` (Lecture 4) is done — today's task is a whole new endpoint, following the pattern already set by `books.py` / `book.py` / `book_service.py`.

In class (Elephant phase, Plan Mode, no code):

1. `/design-discussion` — request/response shape and status codes, in prose
2. Sycophant challenge — what validation is missing?
3. `/write-spec` → `docs/plans/post-reviews.md`

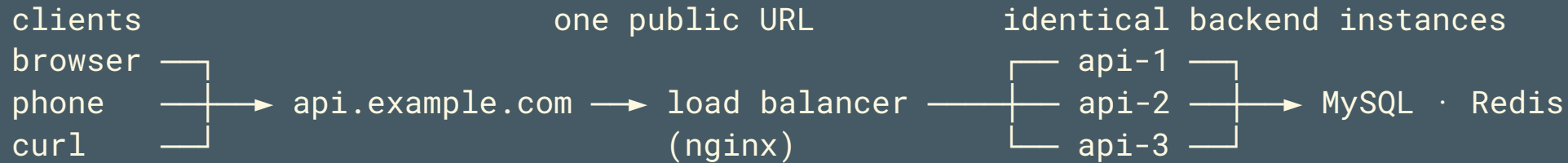
See `project/README.md` for the fixed contract your design should land on.

Activity — Before Next Lecture

- `/goldfish-test docs/plans/post-reviews.md` in a fresh session
- Implement: new `schemas/review.py`, `services/review_service.py`, `routers/reviews.py` — in-memory only, no database yet
- `pytest tests/ -v` — all 19 tests should pass
- Verify via Swagger UI (`/docs`), then `/mean-review`
- Push to GitLab

 **Project 2 released Wed Sep 16** — due Tue Oct 6, 11:59 PM

A Real App Is Not One Server



- Instances get **added** under load, **replaced** on every deploy, and **restarted** when they crash.
- The client never learns which instance answered — and its next request may land on a different one.
- Project 2 runs exactly this shape on your laptop: nginx in front, FastAPI behind, MySQL and Redis shared.

In-Memory = One-Server Design

Your `review_service.py` will keep reviews in a module-level Python list.

- On one instance: correct, all 19 tests pass.
- On two instances: `POST /reviews` lands on `api-1`, the next `GET` lands on `api-2` — the review is gone.

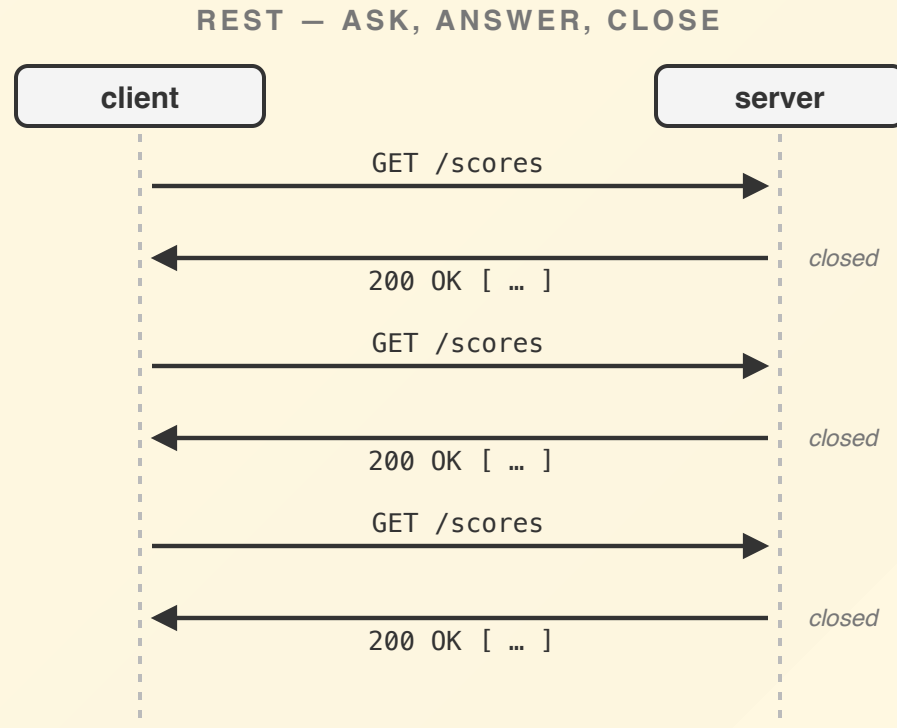
Not a bug in your code — a fact about **where state lives**. That's why Lecture 6 moves it to MySQL.

Why REST Is Stateless

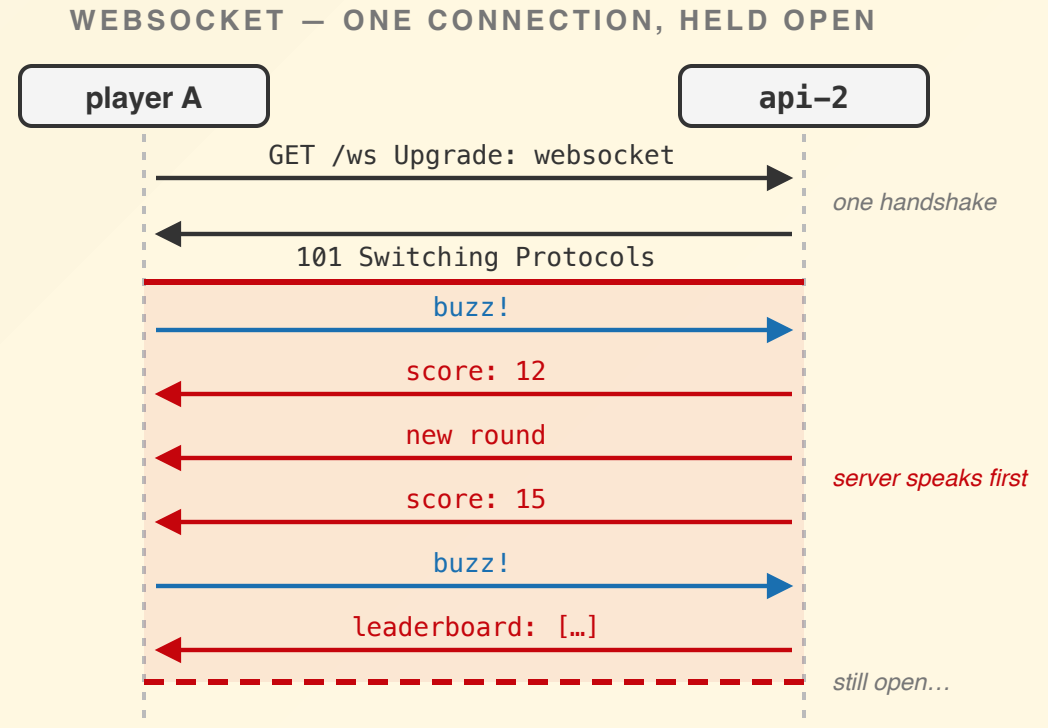
Because *any* instance may answer *any* request, no instance can be the one that "remembers" a client.

- Every request carries everything needed: **which resource** (URL), **what data** (body), **who you are** (a header — Lecture 8's JWT).
- State that must outlive a request lives in **shared storage every instance can reach**: the database (Lecture 6), Redis (Lecture 9).
- The payoff: scaling means adding instances behind the balancer. **Your route code doesn't change.**

WebSockets Hold the Connection



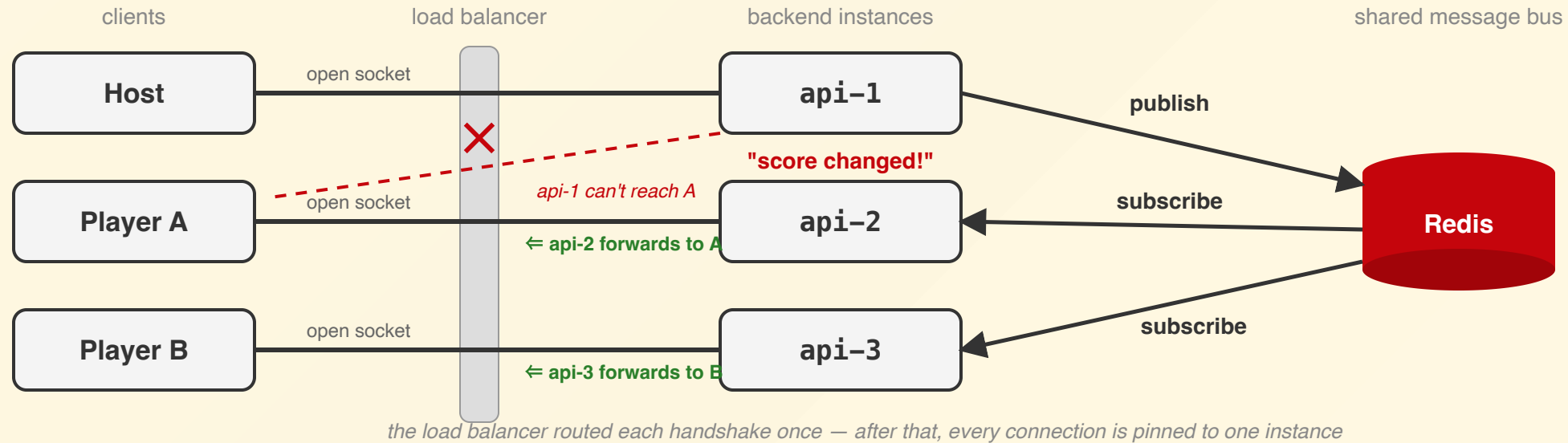
"Any news?" every few seconds.
Between requests the server holds nothing.



Either side, any time — most of it is the server **pushing**.
This socket lives on **api-2** — and api-2 now remembers player A.

One connection, held open. The server can speak first — and **one** server now remembers you.

Many Servers, Pinned Sockets



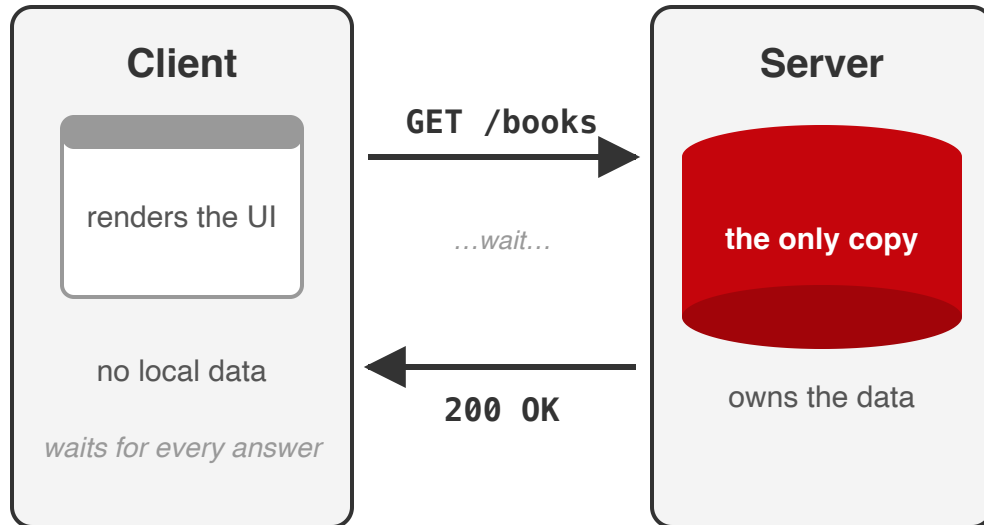
The host's update lands on `api-1`, but Player A is pinned to `api-2`. The fix is a **shared bus**: every instance publishes to Redis and forwards to *its own* sockets — Project 2's live game state.

Buzzer Checkpoint

**Launch Question 7 – discuss answers, then
continue**

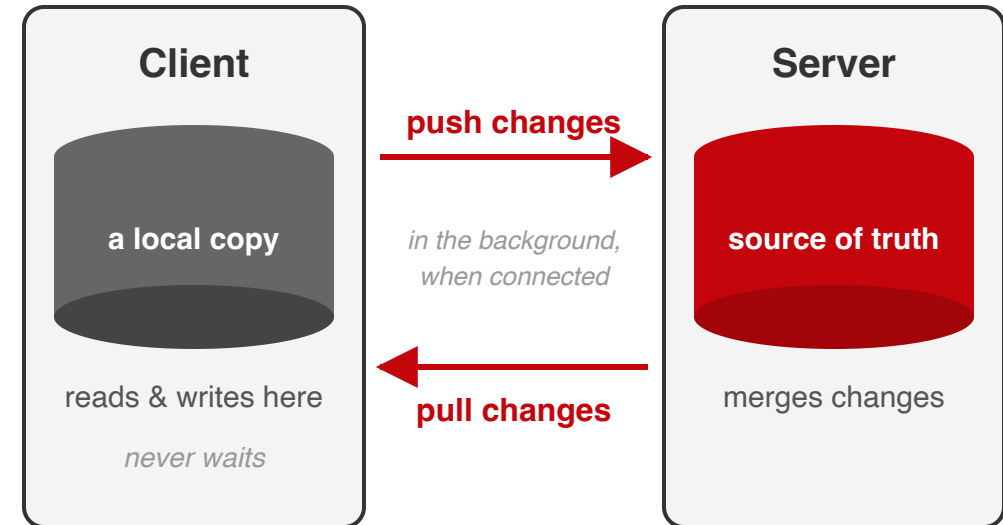
Thin Client vs. Local-First

QUERY — thin client



Every action is a **question** for the server.
No connection → nothing works.

LOCAL-FIRST — full client



Every action is **local**. The connection only
keeps two databases in agreement.

When to Go Local-First

Go local-first when:

- Users keep working with no connection — email on a plane, mobile apps
- Every click must feel instant — no round trip first
- Several people edit the same thing at once — docs, boards, design tools

Stay thin when:

- Data too large or too sensitive to copy to every client
- The server must decide *before* anything appears — payments, grades
- You're learning the contract model — where we are today